

07 — Parallel Query

"Parallel query: when one core is not enough, let PostgreSQL use them all."

Table of Contents

1. [Learning Objectives](#)
 2. [Parallel Query Architecture](#)
 3. [ASCII Diagram: Gather Node and Parallel Workers](#)
 4. [Parallel-Capable Operations](#)
 5. [Configuration Parameters](#)
 6. [Parallel Sequential Scan](#)
 7. [Parallel Index Scan and Index Only Scan](#)
 8. [Parallel Hash Join \(PostgreSQL 11+\)](#)
 9. [Parallel Aggregation](#)
 10. [Parallel Sort and Merge](#)
 11. [Gather vs Gather Merge Nodes](#)
 12. [EXPLAIN Output for Parallel Queries](#)
 13. [Safety and Parallelism Restrictions](#)
 14. [When Parallel Query Doesn't Help](#)
 15. [Common Mistakes](#)
 16. [Best Practices](#)
 17. [Performance Considerations](#)
 18. [Interview Questions & Answers](#)
 19. [Exercises with Solutions](#)
 20. [Production Scenarios](#)
 21. [Cross-References](#)
-

Learning Objectives

After completing this section you will be able to:

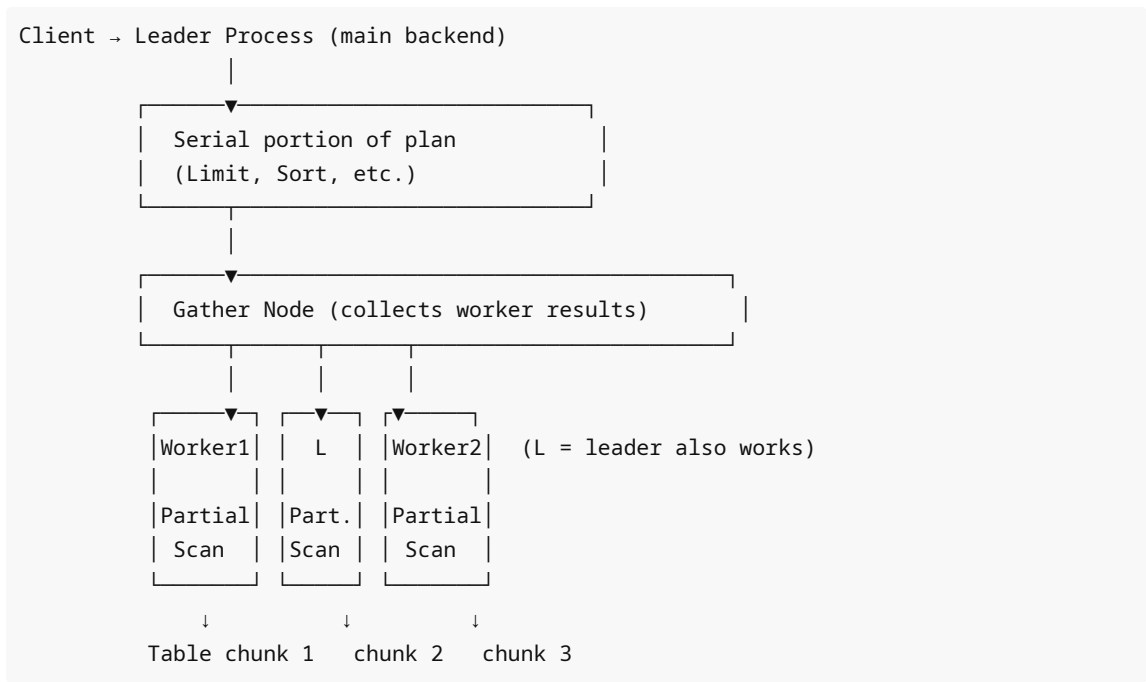
- Explain the parallel query architecture (leader, workers, Gather node)
 - Configure parallel query parameters for optimal performance
 - Read EXPLAIN output for parallel plans including workers and loops
 - Identify which operations can be parallelized
 - Troubleshoot queries that should parallelize but don't
 - Understand safety restrictions that prevent parallelism
-

Parallel Query Architecture

PostgreSQL's parallel query uses a **fork-based model**:

1. The **leader process** processes the query up to parallel-eligible nodes
2. The leader spawns **worker processes** (from a background worker pool)
3. Workers cooperate on the parallel scan/join/aggregate operations
4. A **Gather** or **Gather Merge** node collects results from all workers
5. The leader continues with remaining serial portions of the plan

Process Model



Workers are **background worker processes** that share the same heap pages via `shared_buffers`. No data is copied — workers read directly from the shared buffer pool.

ASCII Diagram: Gather Node and Parallel Workers



Seq Scan	Seq Scan	Seq Scan	Seq Scan
Pages	Pages	Pages	Pages
0..15624	15625..31249	31250..	46875..
		46874	62499

(62,500 pages each, total 250,000 pages)

Each worker:

- Scans 1/4 of the table
- Computes a partial COUNT
- Sends partial count to Gather node

Leader's Finalize Aggregate:

- Receives 4 partial COUNTs
- Sums them → final COUNT
- Returns to client

Speedup: ~3.5x (4 workers, some overhead)

Parallel-Capable Operations

Can Run in Parallel

- Sequential Scan (Parallel Seq Scan)
- Index Scan (Parallel Index Scan , PostgreSQL 11+)
- Index Only Scan (Parallel Index Only Scan , PostgreSQL 11+)
- Hash Join (Parallel Hash Join , PostgreSQL 11+)
- Nested Loop (outer side parallelized)
- Merge Join (outer side parallelized)
- Aggregation (Partial/Finalize Aggregate)
- Sort (Parallel Sort , PostgreSQL 12+)
- Bitmap Heap Scan (PostgreSQL 13+)

Cannot Run in Parallel

- Any query that modifies data (INSERT, UPDATE, DELETE) — except `INSERT INTO ... SELECT`
- Queries using cursors
- Queries with `FOR UPDATE / FOR SHARE`
- Functions marked `PARALLEL UNSAFE` (default for custom functions)
- Queries with `GROUP BY GROUPING SETS , ROLLUP , CUBE` (limited)
- Queries involving temporary tables (pre-PG16)
- Subqueries with `LIMIT` in some cases

Configuration Parameters

Core Parameters

```
-- Maximum workers per Gather node (per query node, not total)
SHOW max_parallel_workers_per_gather; -- default 2
```

```

-- Maximum total parallel workers (across all active queries)
SHOW max_parallel_workers;           -- default 8

-- Cost to set up parallel workers (amortized, reduces parallelism for cheap
queries)
SHOW parallel_setup_cost;             -- default 1000

-- Additional cost per tuple in parallel mode (communication overhead)
SHOW parallel_tuple_cost;             -- default 0.1

-- Minimum table size to consider parallel scan
SHOW min_parallel_table_scan_size;    -- default 8MB
SHOW min_parallel_index_scan_size;    -- default 512kB

-- Enable/disable parallel features
SHOW enable_parallel_query;           -- default on
SHOW enable_parallel_hash;            -- default on (PG11+)

```

Setting for Production

```

-- For OLAP/reporting server (maximize parallelism):
ALTER SYSTEM SET max_parallel_workers_per_gather = 8;
ALTER SYSTEM SET max_parallel_workers = 32;
ALTER SYSTEM SET parallel_setup_cost = 100;    -- lower = parallel for smaller
tables
ALTER SYSTEM SET min_parallel_table_scan_size = '1MB';
SELECT pg_reload_conf();

-- For OLTP server (minimize parallelism to avoid contention):
ALTER SYSTEM SET max_parallel_workers_per_gather = 2;
ALTER SYSTEM SET parallel_setup_cost = 1000;    -- only for large queries
SELECT pg_reload_conf();

```

Forcing/Disabling Parallelism

```

-- Force maximum parallelism for a specific query:
SET max_parallel_workers_per_gather = 8;
SET parallel_setup_cost = 0;
SET min_parallel_table_scan_size = 0;
EXPLAIN SELECT COUNT(*) FROM big_table;

-- Disable parallelism (testing):
SET enable_parallel_query = off;
-- or:
SET max_parallel_workers_per_gather = 0;

```

Parallel Sequential Scan

The table is divided into "chunks" — each worker scans its own chunk.

```
EXPLAIN SELECT COUNT(*) FROM events;
```

```
Finalize Aggregate (cost=1234.56..1234.57 rows=1 width=8)
-> Gather (cost=1000.00..1234.45 rows=4 width=8)
    Workers Planned: 4
    -> Partial Aggregate (cost=234.45..234.46 rows=1 width=8)
        -> Parallel Seq Scan on events
            (cost=0.00..200.00 rows=12500 width=0)
```

- 4 workers planned → each scans ~1/4 of the table
- `Partial Aggregate` computes partial COUNT in each worker
- `Gather` collects partial aggregates
- `Finalize Aggregate` sums them

Heap Scanning

Workers use a **dynamic work distribution**: each worker requests the next heap page from a shared scan position. This ensures even distribution even if page read times vary.

Parallel Index Scan and Index Only Scan

PostgreSQL 11+ can parallelize index scans. Each worker reads its own range of index entries.

```
EXPLAIN SELECT * FROM orders WHERE customer_id BETWEEN 1 AND 1000000;
```

```
Gather (cost=1000.43..45678.90 rows=1000000 width=128)
Workers Planned: 4
-> Parallel Index Scan using idx_orders_customer on orders
    (cost=0.43..11234.56 rows=250000 width=128)
    Index Cond: ((customer_id >= 1) AND (customer_id <= 1000000))
```

Workers split the index range and each scans its portion. The index must be sorted — B-tree naturally supports this split.

Parallel Hash Join (PostgreSQL 11+)

Before PG11: only the outer (probe) side could be parallel. The hash table was built serially by the leader.

From PG11: both sides can be parallel. Workers cooperate to build a shared hash table in parallel, then each worker probes with its portion of the outer table.

```
EXPLAIN SELECT * FROM large_orders lo JOIN customers c ON lo.customer_id = c.id;
```

```
Gather (cost=... Workers Planned: 4)
-> Parallel Hash Join (cost=... rows=... width=...)
    Hash Cond: (lo.customer_id = c.id)
    -> Parallel Seq Scan on large_orders
```

```
-> Parallel Hash      ← All workers build hash table cooperatively
    -> Parallel Seq Scan on customers
```

The `Parallel Hash` node means all workers contribute to building the shared hash table stored in shared memory (not each worker's private memory).

Parallel Aggregation

Parallel aggregation works in two phases:

1. `Partial Aggregate` : each worker computes a partial aggregate over its rows
2. `Finalize Aggregate` (serial): combines all partial aggregates

```
EXPLAIN SELECT status, COUNT(*), SUM(total) FROM orders GROUP BY status;
```

```
Finalize HashAggregate (cost=... rows=3 width=...)
  Group Key: status
    -> Gather (Workers Planned: 4)
      -> Partial HashAggregate (rows=3 width=...)
        Group Key: status
          -> Parallel Seq Scan on orders
```

Each worker computes `{status → partial_count, partial_sum}` for its rows. `Finalize HashAggregate` merges: sum the partial counts, sum the partial sums per group.

Not All Aggregates Parallelize

Simple aggregates (COUNT, SUM, MIN, MAX, AVG) split into partial/finalize phases easily. Some custom aggregates without a `COMBINEFUNC` cannot be parallelized.

Parallel Sort and Merge

PostgreSQL 12+ can sort in parallel using a separate sort per worker, then merge.

```
EXPLAIN SELECT * FROM events ORDER BY user_id, created_at LIMIT 100;
```

```
Limit (rows=100)
  -> Gather Merge (Workers Planned: 4)
    -> Sort (loops=4, each worker sorts its chunk)
      Sort Key: user_id, created_at
        -> Parallel Seq Scan on events
```

Each worker sorts its subset. `Gather Merge` performs a K-way merge of the 4 sorted streams, returning the top 100 rows overall without sorting the entire result.

Gather vs Gather Merge Nodes

Node	Output	Usage
------	--------	-------

Gather	Unordered (random order from workers)	Aggregation, joins where order doesn't matter
Gather Merge	Ordered (merge sorted worker outputs)	ORDER BY, requires workers to produce sorted output

Gather Merge is more expensive (merging 4 sorted streams) but produces ordered output without a final sort, which is essential when combined with LIMIT .

EXPLAIN Output for Parallel Queries

Reading Parallel EXPLAIN

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT COUNT(*), status FROM orders WHERE created_at > '2024-01-01'
GROUP BY status;
```

```
Finalize HashAggregate (cost=12345.00..12345.03 rows=3 width=10)
                        (actual time=456.789..456.790 rows=3 loops=1)
  Group Key: status
  Batches: 1 Memory Usage: 24kB
  -> Gather (cost=11000.00..12344.94 rows=12 width=10)
      (actual time=456.234..456.782 rows=12 loops=1)  ← 3 statuses × 4 workers
= 12
    Workers Planned: 4
    Workers Launched: 4
    -> Partial HashAggregate (cost=10000.00..10000.07 rows=3 width=10)
        (actual time=454.123..454.124 rows=3 loops=4)
                                                ↑ loops=4 (ran 4
times)
      Group Key: status
      Batches: 1 Memory Usage: 24kB
      -> Parallel Seq Scan on orders
          (cost=0.00..9500.00 rows=250000 width=8)
          (actual time=0.034..389.123 rows=250000 loops=4)
                                              ↑ loops=4
          Filter: (created_at > '2024-01-01')
          Rows Removed by Filter: 250000 ← per worker!
          Buffers: shared hit=3125 read=125 ← per worker!
```

Key points:

- loops=4 on inner nodes → ran 4 times (1 leader + 3 workers, but leader counted in loops)
- rows=250000 loops=4 → each worker processed 250K rows; total = 1M rows
- Buffers: shared hit=3125 loops=4 → total buffer hits = 3125 × 4 = 12,500
- The Gather node shows rows=12 — 3 partial aggregates × 4 workers

Safety and Parallelism Restrictions

PostgreSQL requires function safety classification:

```
-- Default for new functions: PARALLEL UNSAFE (prevents parallelism)
CREATE FUNCTION my_func(x INT) RETURNS INT LANGUAGE SQL AS $$
    SELECT x * 2;
$$;
-- This prevents parallel query for any query using my_func!

-- Mark as PARALLEL SAFE if truly safe (no writes, no external state):
CREATE FUNCTION my_func(x INT) RETURNS INT
LANGUAGE SQL PARALLEL SAFE AS $$
    SELECT x * 2;
$$;

-- Or mark as PARALLEL RESTRICTED (can run in parallel leader only):
CREATE FUNCTION get_setting(key TEXT) RETURNS TEXT
LANGUAGE SQL PARALLEL RESTRICTED AS $$
    SELECT current_setting(key);
$$;
```

Safety Classification

Level	Description
PARALLEL UNSAFE (default)	Cannot be executed in parallel workers
PARALLEL RESTRICTED	Can run in parallel leader only (not workers)
PARALLEL SAFE	Can run in any parallel worker

Checking Function Safety

```
SELECT proname, proparallel
FROM pg_proc
WHERE proname = 'your_function_name';
-- proparallel: 'u' = unsafe, 'r' = restricted, 's' = safe
```

When Parallel Query Doesn't Help

- 1. **Short queries:** parallel setup cost (1000 cost units) exceeds savings for small tables
- 2. **Already fast queries:** adding parallelism adds overhead without benefit
- 3. **I/O bound queries:** if all workers compete for the same disk I/O, no speedup
- 4. **Sequential OLTP queries:** parallel query is for analytical workloads
- 5. **Already parallel enough:** if CPU utilization is already at 100% on all cores, adding more workers doesn't help
- 6. **PARALLEL UNSAFE functions:** any unsafe function in the query prevents parallelism

Common Mistakes

1. Not setting `max_parallel_workers_per_gather` high enough

```
-- Default is 2 – too low for large analytical workloads
-- Check how many CPU cores are available:
-- On 16-core server, set to 4-8:
ALTER SYSTEM SET max_parallel_workers_per_gather = 8;
```

2. Not marking custom functions as `PARALLEL SAFE`

Any custom function defaults to `PARALLEL UNSAFE`. This silently prevents parallelism for queries using that function.

3. Setting `work_mem` too high with parallel workers

With 4 workers, each uses `work_mem` memory:

- `work_mem` = 1GB × 4 workers = 4 GB just for hash joins
- Multiply by concurrent connections → OOM risk!
- Use `SET LOCAL work_mem = '256MB'` for specific queries instead of globally

4. Expecting parallelism for OLTP (short) queries

Parallel query adds overhead (process spawning, communication). For queries that run in < 50ms, the overhead often outweighs the benefit.

5. Not considering I/O saturation

If the storage is already at max I/O throughput, adding parallel workers doesn't help and may cause I/O contention.

Best Practices

1. Configure for your workload:

- OLAP: `max_parallel_workers_per_gather = 8`, `parallel_setup_cost = 100`
- OLTP: `max_parallel_workers_per_gather = 2`, `parallel_setup_cost = 1000` (default)

2. Mark custom functions as `PARALLEL SAFE` if they are truly safe (read-only, no external state).

3. Use `EXPLAIN (ANALYZE)` to verify workers are launched: `Workers Launched: N` should match `Workers Planned: N`.

4. Monitor `pg_stat_activity` during large parallel queries to see worker processes.

5. Tune `max_parallel_workers` globally: should not exceed CPU core count × 0.75 to avoid over-subscription.

6. For single large query: can temporarily increase workers:

```
SET max_parallel_workers_per_gather = 16;
SELECT ...;
RESET max_parallel_workers_per_gather;
```

Performance Considerations

Speedup Formula

Theoretical speedup from N workers: $N \times \text{worker_efficiency}$

- Typical efficiency: 0.7-0.9 (overhead from communication, load imbalance)
- 4 workers → ~2.8-3.6× speedup
- 8 workers → ~5.6-7.2× speedup
- Diminishing returns above ~16 workers

Memory

Each parallel worker has its own `work_mem` budget. Total memory for a query with 4 workers and 3 joins: $4 \text{ workers} \times 3 \text{ joins} \times \text{work_mem} = 12 \times \text{work_mem}$.

```
-- Safe work_mem for parallel environment:
-- Total_RAM × 0.5 / max_connections / max_parallel_workers_per_gather
-- e.g., 64GB × 0.5 / 100 connections / 4 workers = 80MB per operation
SET work_mem = '80MB';
```

Interview Questions & Answers

Q1. How does PostgreSQL parallel query work?

A: PostgreSQL spawns background worker processes that share access to the same shared buffer pool. The leader process plans the query and at parallel-eligible nodes, spawns workers. Each worker processes a portion of the data independently (e.g., different heap pages for a Parallel Seq Scan, different index ranges for Parallel Index Scan). Results are collected by a Gather or Gather Merge node in the leader. The parallel portion is a sub-plan; the rest of the query (LIMIT, final aggregate, etc.) runs serially in the leader.

Q2. What is the difference between Gather and Gather Merge?

A: `Gather` collects results from all workers in arbitrary order (first-available). Used when output order doesn't matter (e.g., aggregation). `Gather Merge` collects results from workers that each produce sorted output, then performs a K-way merge to produce globally sorted output. Used when ORDER BY is required. `Gather Merge` is more expensive than `Gather` (merge overhead) but eliminates the need for a final sort step, making it efficient when combined with LIMIT on large sorted datasets.

Q3. Why might a query not use parallel execution even when parallelism is configured?

A: Several reasons: (1) Table is smaller than `min_parallel_table_scan_size` (default 8 MB). (2) Cost benefit is too small compared to `parallel_setup_cost`. (3) Query uses a function marked `PARALLEL UNSAFE` (the default for custom functions). (4) Query modifies data (INSERT/UPDATE/DELETE) except INSERT ...

SELECT. (5) Query uses a cursor. (6) Query uses FOR UPDATE/FOR SHARE. (7)
max_parallel_workers_per_gather = 0 (parallelism disabled).

Exercises with Solutions

Exercise 1

A COUNT query on a 10B row table takes 45 minutes with default settings. Optimize it.

Solution:

```
-- Check current settings
SHOW max_parallel_workers_per_gather; -- probably 2
SHOW max_parallel_workers;           -- probably 8

-- Enable aggressive parallelism for this session:
SET max_parallel_workers_per_gather = 16;
SET max_parallel_workers = 32;
SET parallel_setup_cost = 0;
SET min_parallel_table_scan_size = 0;

-- Run the query and check EXPLAIN:
EXPLAIN (ANALYZE, BUFFERS)
SELECT COUNT(*) FROM huge_table WHERE condition = true;

-- Expected: 16 parallel workers scanning in parallel
-- Speedup: ~10-12x on 16 cores → ~4 minutes instead of 45
```

Production Scenarios

Scenario 1: Analytics Reports Suddenly Slow

An analytics server was using 2 parallel workers (default) for all reports. After the data team grew and started running more concurrent reports, queries slowed down.

```
-- Problem: max_parallel_workers = 8 and 4 simultaneous queries each using 2 workers
-- = 8 workers total, fully utilized → new queries wait

-- Diagnosis:
SELECT pid, query, state, wait_event_type, wait_event
FROM pg_stat_activity
WHERE state = 'active' AND wait_event = 'ParallelWorker';

-- Fix: dedicate more background workers and reduce per-query consumption
ALTER SYSTEM SET max_parallel_workers = 24;
ALTER SYSTEM SET max_parallel_workers_per_gather = 4; -- balanced across queries
SELECT pg_reload_conf();
```

Cross-References

- [01_explain_basics.md](#) — Reading parallel nodes in EXPLAIN
- [02_explain_analyze.md](#) — Loops and timing for parallel nodes
- [04_join_algorithms.md](#) — Parallel Hash Join
- [05_query_planner.md](#) — Parallel cost model